

Rapport de Projet: Architecture Orientée Services (SOA)

Haytham Kennouz

22 Mars 2026

Résumé

Ce rapport détaille la mise en œuvre d'une application e-commerce basée sur l'Architecture Orientée Services (SOA), conformément aux objectifs du TP. Le projet implique la création et l'intégration de plusieurs microservices pour la gestion des produits, des commandes, un serveur de découverte Eureka et une passerelle API. Les fonctionnalités clés telles que la mise à jour des stocks et la gestion des erreurs ont été implémentées avec succès.

1 Introduction

L'objectif de ce Travail Pratique était de concevoir et d'implémenter une application e-commerce simple en utilisant les principes de l'Architecture Orientée Services (SOA) avec Spring Boot. Cette approche permet de créer des services indépendants et réutilisables, facilitant la maintenance et l'évolution de l'application.

2 Architecture du Système

L'application e-commerce est structurée autour de quatre microservices principaux, chacun ayant une responsabilité distincte et une base de données indépendante (H2 en mémoire pour la simplicité du développement).

2.1 Eureka Server

Le serveur Eureka est le composant central de découverte de services. Il permet aux microservices de s'enregistrer auprès de lui et de se découvrir mutuellement. Cela assure une communication flexible et découplée entre les différents services de l'application.

- Port : 8761
- Fonctionnalité : Registre et découverte des microservices.

2.2 Product Service

Ce microservice est responsable de la gestion du catalogue de produits. Il expose des API RESTful pour les opérations CRUD (Créer, Lire, Mettre à jour, Supprimer) sur les produits.

- Port : 8081
- Fonctionnalité : Gestion du catalogue de produits (nom, description, prix, stock).
- Base de données : H2 en mémoire (`productdb`).

2.3 Order Service

Le service de commande gère le cycle de vie des commandes clients. Il interagit avec le service de produits via Feign Client pour récupérer les informations sur les produits et mettre à jour leur stock.

- Port : 8082
- Fonctionnalité : Gestion des commandes clients, interaction avec `Product Service`.
- Base de données : H2 en mémoire (`orderdb`).

2.4 API Gateway

La passerelle API sert de point d'entrée unique pour toutes les requêtes clients externes. Elle achemine les requêtes vers les microservices appropriés (Product Service et Order Service) en fonction des chemins d'accès définis.

- Port : 8080
- Fonctionnalité : Routage des requêtes, équilibrage de charge côté client.

3 Fonctionnalités Implémentées

En plus de la structure de base des microservices, les exercices pratiques suivants ont été mis en œuvre pour enrichir l'application.

3.1 Mise à Jour du Stock (Exercice 1)

Une fonctionnalité a été ajoutée au `Product Service` pour permettre la mise à jour du stock des produits. Le `Order Service` a été modifié pour appeler cet endpoint après la création d'une commande, assurant ainsi que le stock est décrémenté en conséquence. Des vérifications sont effectuées pour s'assurer que le stock est suffisant avant de finaliser une commande.

- `Product Service` : Ajout d'un endpoint PUT `/api/products/{id}/stock` pour décrémenter le stock.
- `Order Service` : Appel à l'endpoint de mise à jour du stock via `FeignClient` lors de la création d'une commande.

3.2 Gestion Améliorée des Erreurs (Exercice 3)

La gestion des erreurs dans le `Order Service` a été améliorée pour fournir des messages d'erreur plus pertinents et des codes de statut HTTP appropriés.

- **Produit introuvable** : En cas de tentative de commande pour un produit inexistant, le système retourne une erreur `400 Bad Request` avec un message clair.
- **Stock insuffisant** : Si la quantité demandée dépasse le stock disponible, une erreur `400 Bad Request` est retournée, informant l'utilisateur du problème.
- **Service produit indisponible** : La résilience du système a été renforcée pour gérer les cas où le `Product Service` est temporairement indisponible. Une `FeignException` est interceptée et transformée en `OrderCreationException`, remontant une erreur significative.

Des `@RestControllerAdvice` ont été implémentés dans les services `Product Service` et `Order Service` pour intercepter les exceptions `RuntimeException` et `OrderCreationException`, et les mapper à des réponses HTTP `400 Bad Request` ou `500 Internal Server Error` selon le cas.

4 Conclusion

Ce TP a permis d'acquérir une compréhension approfondie de l'architecture SOA et de sa mise en œuvre pratique avec Spring Boot. La création de microservices indépendants, leur communication via REST et Feign, la découverte de services avec Eureka, et l'orchestration via une passerelle API ont été des étapes clés. Les exercices ont démontré l'importance de la gestion des stocks et d'une robustesse adéquate dans la gestion des erreurs pour une application distribuée. L'application e-commerce développée est fonctionnelle et respecte les principes d'une architecture microservices.